- `@inject` *services*—You can use dependency injection (DI) to make services available in your views, though this should normally be used sparingly. Using the directive `@inject Service mySService` injects a variable called `mySService` of type `Service` from the DI container, which you can use in your Razor view.

Far and away the best approach for passing data from a page handler to a view is to use properties on the `PageModel`. There's nothing special about the properties themselves; you can store anything there to hold the data you require.

NOTE Many frameworks have the concept of a data context for binding UI components. The `PageModel` is a similar concept, in that it contains values to display in the UI, but the binding is one-directional; the `PageModel` provides values to the UI, and once the UI is built and sent as a response, the `PageModel` is destroyed.

As I described in section 17.2.1, the `@model` directive at the top of your Razor view describes which `Type` of `PageModel` is associated with a given Razor Page. The `PageModel` associated with a Razor Page contains one or more page handlers and exposes data as properties for use in the Razor view, as shown in the following listing.

Listing 17.4 Exposing data as properties on a `PageModel`

```
public class ToDoItemModel : PageModel          ❶
{
    public List<string> Tasks { get; set; }     ❷
    public string Title { get; set; }           ❷

    public void OnGet(int id)
    {
        Title = "Tasks for today";              ❸
```